

Feasibility of Homomorphic Inference for a Genomic Foundation Model: A Client-Assisted CKKS Evaluation of DNAGPT

Christos Galanopoulos¹, Kimon Antonios Provatas¹, Ilias Georgakopoulos-Soares^{1,*}

¹The University of Texas at Austin, Austin, TX, USA

*Correspondence: Ilias Georgakopoulos-Soares

Homomorphic encryption · Genomic privacy · Transformer inference · GPU acceleration

Abstract

Motivation. Genomic sequence identifies its contributor and cannot be replaced once it has been disclosed. When a genomic model is available only as a service, using it ordinarily means handing that sequence to the provider. We ask whether a provider can instead evaluate a genomic transformer without receiving the genomic input in plaintext, and where numerical agreement, memory, or cost would stop such a deployment.

Results. We first reproduce DNAGPT on three genomic task families and record its plaintext predictions as the numerical reference for encrypted evaluation. A fully non-interactive CKKS implementation must approximate the transformer nonlinearities homomorphically. In our evaluated configuration, one transformer block completes, but multi-block composition exceeds the tested 80 GB accelerator-memory envelope. We therefore use a client-assisted protocol: the provider evaluates the transformer on encrypted input representations, while the data owner evaluates the LayerNorm inverse square root, causal softmax, and GELU at predetermined boundaries. The client decrypts only its own intermediate values and returns fresh encryptions. These refreshes reset the multiplicative-depth budget and remove the need for homomorphic bootstrapping in the evaluated protocol. All 12 transformer blocks and the task head execute from encrypted embedded vectors without modifying the model. At the 103-token benchmark—the longest prompt among the encrypted-scope classification tasks—the decrypted prediction matches the plaintext reference. We fixed a relative-error tolerance of 4×10^{-2} before encrypted evaluation. The observed classification-margin error is 8.56×10^{-9} , 6.67 orders of magnitude below that threshold; the exact client-side nonlinear functions contribute no polynomial-approximation error. The maximum multiplicative depth between consecutive refreshes remains 13. Device-wide GPU memory used peaks at 9839 MiB and does not increase across transformer blocks. The complete benchmark takes 1.86 hours on one A100 GPU node.

Conclusions. Client-assisted CKKS makes complete DNAGPT transformer inference from encrypted embeddings feasible within the tested accelerator-memory envelope. Client-side evaluation of the nonlinearities bounds multiplicative depth and avoids homomorphic bootstrapping, while stage-scoped plaintext caching controls host

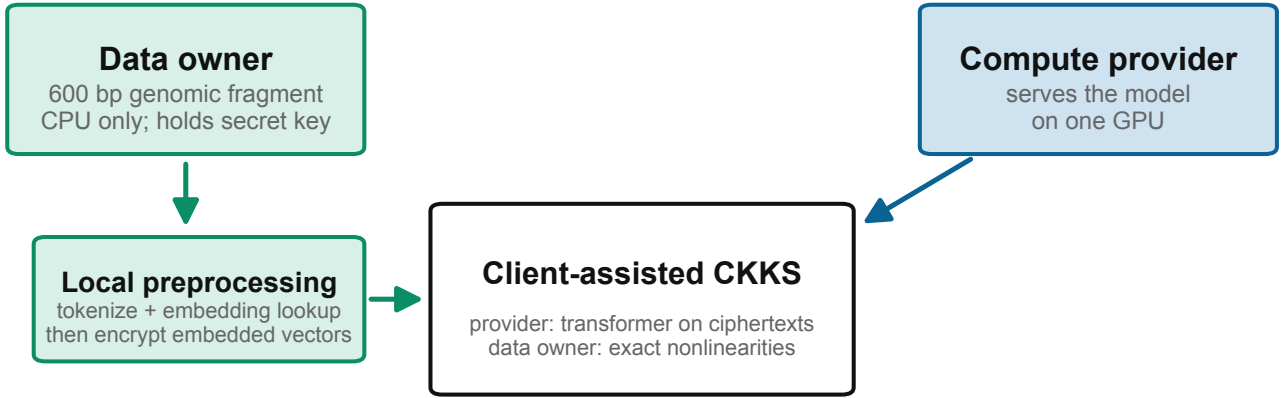
memory. The 1.86-hour runtime of the 103-token benchmark is unsuitable for conventional interactive inference, making provider-side encrypted throughput the principal remaining systems limitation.

Availability. Code, evaluation harnesses, and the evidence record are maintained at https://github.com/Georgakopoulos-Soares-lab/private_genomic_dnagpt.

Key Points

- **The complete model runs on encrypted input.** All twelve transformer blocks and the task head execute at the 103-token benchmark, the longest prompt among the encrypted-scope classification tasks. The numerical tolerance was fixed at 4×10^{-2} before encrypted evaluation; the observed classification-margin error is 8.56×10^{-9} , 6.67 orders of magnitude lower.
- **Encrypted outputs are validated against an independent plaintext reference.** We implement the transformer computation independently in PyTorch float64 and NumPy float64 and compare them after every transformer block and the task head. After agreement is established, the NumPy implementation supplies the reference for encrypted execution.
- **Client refresh bounds multiplicative depth and avoids bootstrapping.** Fresh encryption after each client-evaluated nonlinearity resets the level budget. The maximum multiplicative depth therefore remains 13 throughout the twelve-block model, while device-wide GPU memory used peaks at 9839 MiB without block-over-block growth.
- **The current trade-off favors privacy over latency.** The 103-token benchmark takes 1.86 hours on one A100 and is unsuitable for conventional interactive querying. It may still serve low-volume settings in which a provider retains the model server-side and the data owner accepts higher latency to keep genomic information confidential from a semi-honest provider. Production throughput and networked deployment remain unmeasured.

A genomic fragment stays off the compute provider



The compute provider never receives a plaintext activation or the secret key.

103 tokens benchmark	1.86 h complete benchmark	8.56×10^{-9} vs. float64 reference	14.5% of encrypted evaluation
--------------------------------	-------------------------------------	--	---

Figure 1: The data owner tokenizes and embeds a genomic fragment locally, encrypts the embedded vectors, and retains the secret key. The provider evaluates the transformer on encrypted activations between client boundaries; the data owner evaluates the declared nonlinear functions on CPU. The measured implementation emulates both roles in one process. The result panel reports the complete encrypted execution at the 103-token benchmark, the longest prompt among the encrypted-scope classification tasks.

1 Introduction

Human genomic sequence carries an unusually persistent confidentiality risk. It can identify its contributor and, through familial relationships, reveal information about people who never supplied a sample; re-identification has been demonstrated even after conventional identifiers were removed [Gymrek et al. \[2013\]](#), [Erlich et al. \[2018\]](#). A disclosed genome also cannot be replaced like a password or access token. European data-protection law accordingly treats genetic data as a special category [European Parliament and Council of the European Union \[2016\]](#). Genomic foundation models derive their value from that same sequence. DNAGPT supports classification, regression, and generation through a common transformer backbone [Zhang et al. \[2023\]](#); the downstream tasks evaluated here use human sequence. When such a model is offered only as a service, conventional inference requires the data owner to send the sequence or a representation of it to the provider. Plaintext embeddings do not necessarily remove the risk: per-token representations from DNA foundation models can permit near-perfect sequence reconstruction [Ouaari et al. \[2026\]](#). Homomorphic encryption offers a way for the provider to compute without receiving those genomic values in plaintext.

DNAGPT is publicly available and can be run locally;

public weights make the encrypted computation auditable. The deployment studied here represents a provider that retains a genomic model server-side for commercial, licensing, or governance reasons. In that setting, the data owner must either disclose the genomic input, forgo the inference, or use a privacy-preserving protocol. We therefore ask whether a complete DNAGPT transformer can be evaluated under homomorphic encryption, and what limits its numerical fidelity, memory use, and runtime. Feasibility asks whether the complete computation executes within the cryptographic and memory budget while meeting a predeclared numerical criterion. Practicality asks whether the resulting cost suits the intended deployment.

CKKS supports the packed additions, multiplications, and rotations used by transformer linear layers and attention products [Cheon et al. \[2017\]](#), but LayerNorm, softmax, and GELU require functions that are expensive to evaluate homomorphically. A fully non-interactive design must approximate them with polynomial circuits and provide a larger depth and refresh budget. In our evaluated non-interactive configuration, one transformer block completes, but multi-block composition exceeds the tested memory envelope. This motivates the client-assisted protocol evaluated here: the data owner evaluates the three nonlinear functions exactly at predeter-

mined boundaries and returns fresh ciphertexts. The refreshes reset the multiplicative-depth budget and avoid homomorphic bootstrapping in the retained protocol. Client-assisted nonlinear evaluation is established prior art [Chen et al. \[2022\]](#), [Biswas et al. \[2025\]](#); our study measures its numerical behavior, memory, interaction, and executed work for a complete genomic transformer whose weights and nonlinear formulas remain unchanged.

We first reproduce DNAGPT on genomic-signal recognition, promoter and splice-site classification, and mRNA abundance regression. For numerical validation, we implement the forward computation independently in PyTorch float64 and NumPy float64 and compare their outputs after every transformer block and the task head. Once these implementations agree, the NumPy output becomes the plaintext reference for encrypted evaluation. We then evaluate non-interactive and client-assisted CKKS designs under checked operation schedules.

The client-assisted protocol executes all twelve transformer blocks and the task head at the 103-token benchmark while maintaining a maximum multiplicative depth of 13 and bounded accelerator memory. Its output satisfies the predeclared numerical tolerance by several orders of magnitude. The complete execution takes 1.86 hours on one A100, making runtime instead of correctness or accelerator capacity the main limitation of the current implementation.

The paper makes the following contributions:

- **Complete-model graph feasibility.** All twelve DNAGPT transformer blocks and the task head execute at the 103-token maximum encrypted-scope classification benchmark from encrypted embedded vectors, with the original weights and nonlinear formulas and without homomorphic bootstrapping. To our knowledge this is the first complete CKKS forward pass of a pretrained genomic foundation model; prior homomorphic transformer evaluations over biological sequence data have targeted small, purpose-trained classifiers [Harmanci et al. \[2024\]](#), [Chen et al. \[2024\]](#).
- **Independently checked numerical evaluation.** The protocol, packing, checked operation schedule, predeclared tolerance, and validation chain make the complete execution independently inspectable.
- **Measured resource behavior.** We report complete-model cost, interaction, multiplicative depth, and memory; the stage-bounded cache that makes host-side reuse executable; and the memory boundary reached by the evaluated non-interactive configuration.

2 Background

2.1 DNAGPT and the genomic task families

DNAGPT is an autoregressive transformer adapted to sequence and numerical tasks over DNA [Zhang et al.](#)

[\[2023\]](#). We evaluate its 0.1-billion-parameter configuration. The backbone has 12 transformer blocks, hidden width 768, and 12 causal-attention heads of width 64. Its feed-forward sublayer expands to width 3,072. DNA is represented through 6-mer tokenization and combined with task-format tokens before entering the transformer.

Each block follows the structure that determines the encrypted circuit: LayerNorm; query, key, and value projections; causal self-attention; an output projection and residual; a second LayerNorm; then an expanded MLP with GELU and another residual. The dense maps apply fixed model weights. LayerNorm and GELU apply nonlinear functions to the activations, while attention introduces products between two data-dependent quantities and normalizes them with a softmax.

The release includes fine-tuned heads for genomic-signal recognition and mRNA abundance regression, but none for the genome-understanding benchmark, so Section 4 evaluates that backbone with a locally fine-tuned linear head. All three task families predate DNAGPT and carry established task-specific baselines [Kalkatawi et al. \[2019\]](#), [Agarwal and Shendure \[2020\]](#), [Zhou et al. \[2024\]](#), against which Section 4 establishes model fidelity; the encrypted study then uses the signal-recognition path as its task-length anchor.

2.2 CKKS

CKKS supports approximate arithmetic on packed vectors of real or complex values [Cheon et al. \[2017\]](#). Encoding places many values into the slots of one plaintext, which is then encrypted under the data owner’s key. Addition and multiplication act slotwise across the packed vector, while rotations move values between slots. This single-instruction, multiple-data model is a natural match for transformer activations: a rotation-and-diagonal schedule can express a public matrix transform as ciphertext–plaintext multiplication and accumulation.

Not every transformer product has a public operand. Query–key scoring multiplies two encrypted projections of the input, and attention context multiplies encrypted attention weights by encrypted values. Those operations require ciphertext–ciphertext multiplication and relinearization. The distinction is about data ownership rather than implementation convenience: public weights and masks remain plaintext, while every query-derived operand remains encrypted at the server.

CKKS controls approximate fixed-point scale through rescaling. A multiplication consumes a level from a finite modulus chain, and the computation fails once that multiplicative budget is exhausted. Bootstrapping can refresh a ciphertext without decryption, while fresh client encryption can reset the budget in an interactive protocol. The latter is used by the client-assisted protocol in this study, which therefore requires no homomorphic bootstrapping. Both choices carry system costs, so multiplicative depth is an architectural resource rather than only a cryptographic parameter.

2.3 Why the nonlinearities are the architecture

CKKS directly supplies polynomial arithmetic, but a transformer requires inverse square root in LayerNorm, exponentiation and normalization in softmax, and GELU. A non-interactive circuit must replace these functions with polynomial approximations over declared input ranges. Higher-degree approximations consume additional multiplicative depth; across repeated blocks, the evaluator must provide a deeper context, a bootstrapping schedule, or both. Larger modulus chains and refresh operations also increase the context and evaluation-key material that must remain resident.

The alternative is to place exact functions at declared client boundaries, which removes approximation-range calibration and resets the encrypted depth budget but requires an online client and exposes intermediate activations to it. The choice between polynomial approximation and client evaluation therefore determines memory, interaction, depth, and the threat model. Section 5 evaluates both designs and reports where each stops.

3 Deployment scenario and threat model

3.1 Deployment roles and privacy objective

The deployment has two parties but one cryptographic confidentiality objective. The data owner holds a human genomic sequence, ordinary CPU resources, and no GPU. It will not disclose the sequence to the compute provider. The model owner operates the accelerator infrastructure and serves the DNAGPT model rather than distributing it to clients. Keeping the weights server-side is an operational preference of this deployment, not a model-confidentiality guarantee supplied by the protocol.

The protected asset is the genomic input. The data owner encrypts its embedded numeric vectors and retains the secret key; the compute provider evaluates the served model on those ciphertexts. The evaluated input path assumes that the provider distributes the tokenizer and embedding table so that the data owner can construct those vectors locally, which is compatible with the public artifact used here but is an explicit service assumption: a provider unwilling to distribute those components would need a private token-index lookup, which is outside the evaluated protocol.

3.2 Local inference as a comparison

Local plaintext inference is preferable whenever the client may hold and run the model. It avoids all cryptographic work, and client assistance offers no computational advantage over it; in the complete encrypted inference of Section 7 the data owner alone spends 956.187 s evaluating LayerNorm inverse square roots, causal softmax, and GELU at the boundaries, and this study records no comparable local CPU forward-pass time. Where the client may not hold the weights, the choices are to disclose the

genome, decline the inference, or use a privacy-preserving protocol. Client-assisted CKKS targets deployments in which the provider retains the model server-side and the data owner prioritizes genomic confidentiality over latency. Under the semi-honest threat model, the provider performs inference without receiving the genomic input or intermediate activations in plaintext. The protocol is not intended to outperform local plaintext inference.

The protocol assigns transformer operations on encrypted activations between client boundaries to the party with accelerator hardware, while the key holder evaluates the exact nonlinearities on CPU. This allocation protects the genomic input without requiring the data owner to operate a GPU.

3.3 Adversary model

We consider a semi-honest (honest-but-curious) compute provider under 128-bit classical CKKS parameters. It follows the prescribed circuit but may inspect everything it legitimately receives: public and evaluation keys, model parameters, ciphertexts, sequence length, message sizes, operation order, boundary schedule, and timing. It does not alter ciphertexts to probe the client or deviate from the declared boundary sequence.

Under this adversary, the compute provider receives no plaintext genomic vector, plaintext activation, partial decryption, or secret-key material. At a boundary it supplies a ciphertext and receives a fresh ciphertext under the data owner’s public key. The guarantee concerns the provider’s view of query-derived values; public model structure and plaintext weights remain available to the provider as inputs to ciphertext–plaintext operations.

Invariant 1 (Key custody). *The data owner generates and retains the secret key. No protocol message contains the key or a partial decryption, and the compute provider has no decryption operation.*

Invariant 2 (No plaintext at the server). *Every query-derived value held by the compute provider is a ciphertext under the data owner’s key. Client-evaluated nonlinearities return through fresh encryption rather than as decoded values.*

Invariant 3 (Non-adaptive boundaries). *The circuit and sequence length determine every boundary before execution; decrypted content does not select the next operation. The implementation validates the complete schedule and boundary counts for each accepted run and aborts on deviation. The boundary schedule is therefore fixed by public parameters rather than by the private input, though no separate leakage experiment compares transcripts across different inputs of the same length.*

3.4 What is not claimed

Model confidentiality is not claimed. The data owner observes full intermediate activations at every nonlinearity. This transcript creates a chosen-query extraction surface,

Table 1: Resource and trust allocation under local and served-model deployment. The plaintext service is the ordinary baseline that the encrypted protocol is intended to change.

	Client-local	Plaintext service	Client-assisted CKKS
Client must hold the full model	yes	no	no
Server receives genomic input	no	in plaintext	encrypted vectors
Client requires a GPU	deployment-dependent	no	no
Server evaluates transformer operations	no	in plaintext	on ciphertexts
Client evaluates model nonlinearities	locally	no	at boundaries

and sufficiently diverse input–output pairs may make some affine segments identifiable. No extraction experiment or model-confidentiality proof is provided. Keeping weights on the server may deter casual copying, but it is not a cryptographic confidentiality guarantee. This limitation does not change the genomic-data objective, which protects the data owner from the compute provider.

Malicious servers, authenticated transport, production key custody, traffic analysis, timing and physical side channels, and compromised clients are outside the evaluated boundary. Encrypted scope begins after token-index lookup, so private lookup is also unresolved. Per-token embeddings remain sensitive: high-fidelity sequence reconstruction has been demonstrated for embeddings from other DNA foundation models [Ouaari et al. \[2026\]](#). The embedded vectors are therefore encrypted here rather than sent in clear. The data owner sees its own intermediate activations and final output; privacy from the key holder is not an objective of this protocol.

4 Plaintext validation

Before encrypted evaluation, we reproduced DNAGPT on genomic-signal recognition, promoter and splice-site classification, and mRNA abundance regression. We also recorded the plaintext outputs required for subsequent numerical comparison with CKKS execution.

The upstream DNAGPT implementation is used without changing its forward computation. Published or reconstructed evaluation splits are used for the selected tasks. Genomic-signal recognition is evaluated on the full balanced source corpus, including examples used to train its head, so this result measures model and pipeline reproduction rather than held-out generalization.

Numerical validation uses two independent float64 implementations. A PyTorch execution and a separate NumPy implementation are compared after every transformer block and the task head, with a 2×10^{-5} agreement criterion. After this check passes, the NumPy implementation provides the plaintext reference for encrypted evaluation. The original float32 execution is retained only to quantify dtype-related drift.

Across the selected tasks, the reproduced results retain the expected task signal and provide the model-validation baseline used before encrypted evaluation. The GUE tasks use locally trained heads because DNAGPT provides no corresponding fine-tuned head, and the recov-

ered abundance split is used only as model-fidelity evidence.

4.1 Benchmark prompt length

Genomic-signal recognition uses 600-base-pair inputs. DNAGPT tokenizes each input into 100 6-mer sequence tokens plus three task-format tokens, yielding a 103-token prompt. This is the longest prompt among the encrypted-scope classification tasks and is therefore used as the maximum-length encrypted benchmark in Sections 5 and 7.

5 Client-assisted CKKS protocol

5.1 Why client assistance is needed

A fully non-interactive CKKS transformer must replace the LayerNorm inverse square root, softmax, and GELU with homomorphic approximations. Our evaluated non-interactive configuration completes one transformer block, but the deeper modulus and evaluation-key requirements needed for multi-block composition exceed the tested 80 GB accelerator-memory envelope before composition arithmetic begins. We therefore use client assistance to evaluate these nonlinearities exactly and return fresh ciphertexts. This resets the multiplicative-depth budget and avoids homomorphic bootstrapping in the retained protocol. The non-interactive result is specific to the evaluated library, parameters, packing, and hardware.

5.2 Client nonlinearity boundaries

CKKS carries the affine maps, reductions, residual additions, and attention products without exposing an activation to the compute provider. We place predetermined client boundaries at the three nonlinearity classes. For LayerNorm, the provider computes the encrypted mean and variance and sends only the variance term needed for the inverse square root. For attention, it sends encrypted causal score tiles and receives encrypted softmax weights. The GELU boundary operates on the packed MLP activations.

At each boundary, the data owner decrypts the relevant intermediate value, evaluates the function in double precision, and returns a fresh encryption. Only the data owner

Table 2: Plaintext DNAGPT validation on three genomic task families. Signal recognition and abundance regression use the released fine-tuned heads; the understanding-benchmark rows use a locally fine-tuned head because DNAGPT provides no corresponding head.

Task	Metric	This work	Reference	Source
Polyadenylation-signal recognition [‡]	accuracy	0.9124	0.9151	Zhang et al. [2023]
	F1	0.916	—	
Core promoter detection	MCC	0.680	0.69	Zhou et al. [2024]
300 bp promoter detection	MCC	0.897	0.87	Zhou et al. [2024]
Splice-site classification	MCC	0.831	0.85	Zhou et al. [2024]
mRNA abundance regression ^{†§}	r^2	0.562	0.62	Zhang et al. [2023]
	Pearson r	0.753	—	

[‡]The reference is DNAGPT’s own reported accuracy for this model class, measured on a held-out quarter of the set; this work evaluates all 22,604 examples. The comparison is same-model but not same-split.

[†]Model-fidelity evidence only. This task is outside the encrypted scope; see Section 9.

[§]The released regression head evaluated here is fine-tuned from the human-pretrained 0.1B variant, while the reference is reported for the mammal-pretrained variant, so the backbones differ.

Table 3: Notation. All values are those of the evaluated configuration.

Symbol	Value	Meaning
D	768	DNAGPT hidden width
H	12	attention heads, of $D/H = 64$ dimensions each
D_{mlp}	3072	MLP expansion width, $4D$
T	103	maximum encrypted-scope classification prompt length
B	8	tokens packed per ciphertext (token lanes)
G	13	ciphertext groups, $\lceil T/B \rceil$
W	1024	feature width padded to a power of two
C	4	activation copies per ciphertext, D_{mlp}/D
N	65536	polynomial ring degree
$N/2$	32768	usable plaintext slots per ciphertext, $C \cdot W \cdot B$
L	13	multiplicative depth
λ	128	classical security level, in bits

holds the secret key. The compute provider therefore receives neither the plaintext genomic input nor plaintext intermediate activations. Fresh encryption restarts the CKKS level budget, so the protocol does not require homomorphic bootstrapping. Figure 2 shows the evaluation order for one transformer block and the party responsible for each step.

5.3 Protocol algorithms

Fresh encryption in Algorithm 2 returns the activation to level 0, so every client boundary resets the multiplicative budget. The required parameter set is therefore determined by the maximum multiplicative depth between consecutive refreshes. In the evaluated circuit this maximum is 13, and it does not accumulate across the 12 transformer blocks.

The boundary schedule is determined entirely by the public circuit structure and sequence length, never by de-

encrypted content. The implementation validates the operation schedule, boundary counts, and remaining multiplicative depth on every run and aborts when any differs.

5.4 Packing

Each ciphertext has 32,768 usable slots, indexed by activation copy, padded feature, and token lane. The layout assigns these slots as 4 copies by 1,024 features by 8 token lanes, with the token lane as the innermost index. Rotations by whole-lane strides can therefore move features without mixing tokens, while smaller rotations align tokens within a group. The DNAGPT hidden width of 768 occupies part of the padded feature region; the remainder supplies the power-of-two width required by the linear transform and staging space for per-head attention scores. In dense transforms, rows and columns outside the active width contribute zeros; padding therefore affects cost without changing the numerical result.

The four copies match the MLP expansion from width 768 to 3,072. They let its expansion chunks occupy parallel regions of one ciphertext rather than requiring a separate wide layout. At the 103-token benchmark, the tokens fill 13 ciphertext groups, with padding in the final group. Within a dense transform, a 32×32 baby-step giant-step decomposition covers the 1,024 padded features; baby rotations are shared where the schedule permits. Packing eight tokens together reduces the number of dense encrypted products from 1,236 serial-equivalent products to 156, a structural reduction of $7.92\times$. No latency factor follows from that operation count alone.

5.5 Causal attention

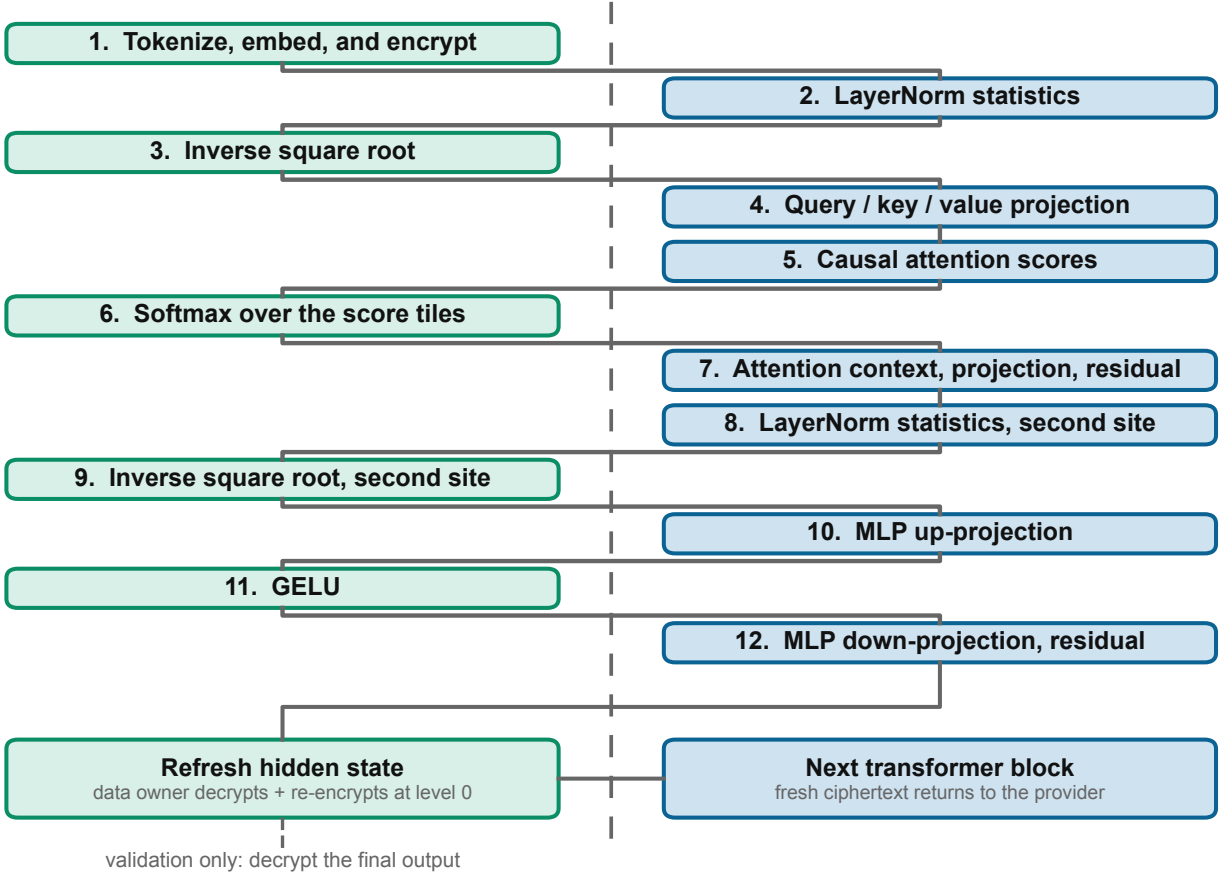
Attention follows a chunked lower-triangular schedule over the 13 token groups. Only causal group pairs are visited, producing 91 encrypted score tiles. Shifts needed by several later query groups are materialized per key group and reused. The client collects all score tiles before applying stable softmax over complete causal rows; normalization is therefore a global synchronization point

Data owner

holds the genomic fragment and secret key · no GPU

Compute provider

holds the model · encrypted activations only · one A100



Fresh encryption resets the CKKS depth budget; no homomorphic bootstrapping is used.

Figure 2: Evaluation order for one transformer block. The provider performs transformer operations on encrypted activations, while the data owner evaluates the LayerNorm inverse square root, causal softmax, and GELU at fixed boundaries and returns fresh ciphertexts. These refreshes reset the multiplicative-depth budget before subsequent encrypted computation.

in this design. It then emits the encrypted weight tiles consumed by the context computation.

Plaintext model weights do not eliminate ciphertext-ciphertext multiplication inside attention: both query-key scoring and softmax-weight-value aggregation multiply one activation-derived quantity by another. Of the block’s 1,506 ciphertext-ciphertext multiplications, LayerNorm contributes 26 centered-value squares and 26 products with client-returned inverse square roots. Attention contributes 727 query-key products and 727 weight-value products, accounting for 97% of the total. The latter term follows directly from returning exact attention weights as fresh ciphertexts; it is the arithmetic price paid to keep the attention nonlinearity exact at the client boundary.

5.6 Depth and parameters

The retained CKKS context uses ring dimension 65,536, yielding 32,768 usable slots at a 128-bit classical security

level. It uses 50-bit scaling, a 60-bit first modulus, 3 hybrid key-switching digits, and 80 rotation keys. Multiplicative depth 13 is the smallest demonstrated passing depth for this scale, digit count, ring, and packing. The block completes with 6 of the 13 levels remaining. The retained setting is the demonstrated feasible point in this parameter space; parameter optimization is a separate experiment. CKKS contexts are constructed with OpenFHE, while encrypted server-side operations use the evaluated CUDA backend Al Badawi et al. [2022], Agulló-Domingo et al. [2025].

5.7 Composition across blocks

Composition uses a declared full-hidden-state client refresh between blocks. The data owner decrypts the packed block output, restores its token-vector layout, and encrypts fresh level-0 inputs for the next block under the same context and key lineage. The complete evaluation uses that mechanism throughout, with 11 full-hidden-state refreshes and one last-token refresh into the task

Algorithm 1 Evaluating one transformer block. $G = 13$ token groups; $\odot$ is homomorphic multiplication; every operation outside a `CLIENT` call runs on ciphertexts.

Require: encrypted activations $c_0, \dots, c_{G-1}$; public weights W

Stage 1: normalization and projection

```

1: for  $g \leftarrow 0$  to  $G - 1$  do
2:    $n \leftarrow \text{LAYERNORM}(c_g, \gamma_1)$  ▷ Algorithm 3
3:    $\beta \leftarrow \text{BABYROTATIONS}(n)$  ▷  $N_1 - 1$  rotations, shared
4:    $Q_g, K_g, V_g \leftarrow \text{MATMUL}(\beta, W_q), \text{MATMUL}(\beta, W_k), \text{MATMUL}(\beta, W_v)$ 
5: end for
6: FLUSHENCODEDWEIGHTS ▷ bounds host memory to one stage

```

Stage 2: causal attention scores

```

7: for  $k \leftarrow 0$  to  $G - 1$  do
8:    $\tilde{K} \leftarrow \{\text{LANESHIFT}(K_k, \delta)\}$  for each  $\delta$  required by a later query group
9:   for  $q \leftarrow k$  to  $G - 1$  do
10:     $S \leftarrow 0$ 
11:    for  $\delta \in \text{ACTIVESHIFTS}(q, k)$  do
12:       $P \leftarrow Q_q \odot \tilde{K}_\delta$ 
13:      for  $h \leftarrow 0$  to  $H - 1$  do
14:         $S \leftarrow S + \text{REDUCESUM}(P \odot \text{mask}_h) \odot \text{isolate}(q, k, \delta, h) / \sqrt{d_h}$ 
15:      end for
16:    end for
17:    CLIENT.TAKESCORETILE( $S, q, k$ ) ▷ decrypt only
18:  end for
19: end for
20: CLIENT.SOFTMAX ▷ stable softmax over complete causal rows

```

Stage 3: attention context

```

21: for  $k \leftarrow 0$  to  $G - 1$  do
22:    $\tilde{V} \leftarrow \{\text{LANESHIFT}(V_k, \delta)\}$ 
23:   for  $q \leftarrow k$  to  $G - 1$ ;  $\delta \in \text{ACTIVESHIFTS}(q, k)$  do
24:      $A \leftarrow \text{CLIENT.EMITWEIGHTTILE}(q, k, \delta)$  ▷ re-encrypt only
25:      $C_q \leftarrow C_q + A \odot \tilde{V}_\delta$ 
26:   end for
27: end for

```

Stages 4 and 5: projection, MLP, residuals

```

28: for  $g \leftarrow 0$  to  $G - 1$  do
29:    $R_g \leftarrow c_g + \text{MATMUL}(\text{BABYROTATIONS}(C_g), W_{\text{proj}})$ 
30: end for
31: FLUSHENCODEDWEIGHTS
32: for  $g \leftarrow 0$  to  $G - 1$  do
33:    $\beta \leftarrow \text{BABYROTATIONS}(\text{LAYERNORM}(R_g, \gamma_2))$ 
34:    $h \leftarrow \sum_{c < 4} \text{MATMUL}(\beta, W_{\text{fc}}^c) \odot \text{copy}_c$ 
35:    $a \leftarrow \text{CLIENT.GELU}(h)$ 
36:    $c'_g \leftarrow R_g + \sum_{c < 4} \text{MATMUL}(\text{BABYROTATIONS}(a \odot \text{copy}_c), W_{\text{mlp}}^c)$ 
37: end for
38: return  $c'_0, \dots, c'_{G-1}$ 

```

Algorithm 2 The client boundary. The data owner is the only party holding sk , and every value it decrypts derives from its own query.

```

1: function CLIENTBOUNDARY( $c, f$ )
2:    $v \leftarrow \text{DECODE}(\text{DECRYPT}(\text{sk}, c))$ 
3:    $v' \leftarrow f(v)$  ▷ evaluated exactly, in double precision
4:   return ENCRYPT( $\text{pk}, \text{ENCODE}(v')$ ) ▷ fresh ciphertext at level 0
5: end function

```

head; Section 7 reports the measured outcome. Fresh level-0 ciphertexts prevent multiplicative depth from accumulating over all twelve blocks.

6 Host-side systems work at fixed circuit

Reusing encoded weight diagonals removes redundant host encoding, but retaining every encoded plaintext causes host memory to accumulate across stages. An unbounded-cache implementation was terminated by the operating system at 61,863 MiB when the MLP began building templates on top of earlier stages. The retained

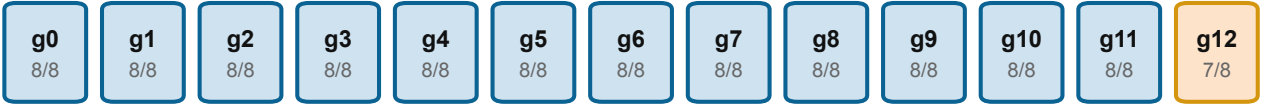
One ciphertext holds 32,768 slots

4 copies $\times$ 1,024 features $\times$ 8 token lanes = 32,768 slots. The copies carry the 4 $\times$ MLP expansion.



103-token genomic-signal benchmark: 600 bp $\rightarrow$ 13 ciphertext groups

600 bp $\div$ 6-mer = 100 tokens + 3 task specials = 103. Longest encrypted-scope classification prompt.
8 token lanes give 13 groups; the final group has one padded lane.



The 13 groups give 91 lower-triangular causal score tiles, and just 1 of the 104 lanes is padding.

Figure 3: Slot layout for the 103-token genomic-signal benchmark, the longest encrypted-scope classification prompt. Each ciphertext carries four copies of 1,024 features across eight token lanes: 768 features are active and the remainder supplies power-of-two padding and attention staging. The benchmark occupies 13 groups and generates 91 lower-triangular causal score tiles.

Algorithm 3 LayerNorm. The server computes both statistics under encryption; only the inverse square root crosses the boundary. The DNAGPT checkpoint carries no bias term.

```

1: function LAYERNORM( $x, \gamma$ )
2:    $\mu \leftarrow \text{REDUCESUM}(x) \odot (1/D)$ 
3:    $z \leftarrow x - \mu$ 
4:    $\sigma^2 \leftarrow \text{REDUCESUM}(z \odot z) \odot (1/D) + \varepsilon$ 
5:    $r \leftarrow \text{CLIENTBOUNDARY}(\sigma^2, t \mapsto t^{-1/2})$ 
6:   return  $(z \odot r) \odot \gamma$ 
7: end function

```

implementation caches encoded diagonals only for the stage in which they are reused, so the largest live stage determines peak host memory.

All retained host-side optimizations preserve the encrypted operation schedule, packing, boundary order, multiplicative depth, and numerical acceptance criterion. Because no dedicated-node measurement of the pre-optimization implementation is available, we report verified changes in executed work and memory instead of a latency speedup.

6.1 Host-side encoding redundancy

The baseline rebuilt and encoded every packed diagonal for every dense product. Its 156 products each required 1,024 diagonal plaintexts, producing 159,744 encoding op-

Algorithm 4 Packed matrix product, baby-step giant-step over the packed diagonals. $N_1 = N_2 = 32$ and $N_1 N_2 = W = 1024$.

```

1: function MATMUL( $\beta, M$ )
2:    $d \leftarrow \text{ENCODEDDIAGONALS}(M, \text{level}(\beta_0))$   $\triangleright$ 
   encoded once, cloned per use
3:    $y \leftarrow 0$ 
4:   for  $j \leftarrow 0$  to  $N_2 - 1$  do
5:      $u \leftarrow \sum_{i < N_1} \beta_i \odot \text{CLONE}(d_{N_1 j + i})$ 
6:      $y \leftarrow y + (j = 0 ? u : \text{ROT}(u, BN_{1j}))$ 
7:   end for
8:   return  $y$ 
9: end function

```

erations per block, and about 90% of that work was byte-identical: the block has 12 weight matrices, each used across all 13 token groups. The GPU could not begin a multiplication until the host had prepared its plaintexts.

In the baby-step giant-step transform, a plaintext diagonal is not merely a view of the original matrix: it must be packed, scaled, and encoded for the ciphertext level at which multiplication will occur. Repeating that conversion for every token group made a model-invariant representation step dominate the call path. The optimization target is therefore the lifetime and placement of plaintext preparation, while the encrypted products and accumulations remain fixed.

6.2 Encode once, clone per use

The retained cache encodes each distinct weight diagonal into a pristine host template. A multiplication receives a lightweight clone that shares the encoded host data by reference while carrying fresh device state; the clone is loaded and evicted for that operation. This removes 90% of the 159,744 encoding operations without keeping a plaintext object resident on the GPU between multiplications.

Cloning is required by the evaluated backend’s object semantics. Its plaintext-load path returns early when an object is already marked as loaded, so directly reusing a template across multiplicative levels can pair a new ciphertext level with stale residue-number-system limbs. Fresh device state on each clone forces a load at the level required by the current operation. An isolated GPU micro-test checked clone-versus-fresh-encode equality, cross-level reuse, stable device memory under repeated multiplication, and equality between parallel and serial encoding before the cache was applied to the complete block.

Templates are indexed by both weight matrix and multiplicative level, which preserves the backend’s level contract while still sharing work among token groups that reach the same matrix at the same level. The cache therefore stores reusable encoded data, not a ciphertext or a mutable GPU object, and leaves homomorphic multiplication itself unchanged.

6.3 Bounding host memory

Removing repeated encoding exchanges computation for host memory. An early cache retained templates from every completed stage, and the process did not survive the MLP. Two exact changes bound the cache. First, a template is encoded directly at its multiplicative level of use, so it carries only the limbs needed by the receiving ciphertext; this changes no arithmetic, because the backend would otherwise drop a level-0 plaintext to the same level before multiplication. Second, the implementation flushes the cache after query/key/value projection and after the attention projection, which are the final use sites for their respective weight sets: no weight matrix recurs across either boundary, so retaining those templates cannot produce another hit.

Within a stage, the group-major loop revisits its weights as it advances through token groups, so early eviction would discard real reuse. Across stages, the next operation consumes a different set of weights, so eviction releases memory without creating a later re-encoding. The target-process resident-set trace follows this policy, as Figure 4(d) shows. Query/key/value projection reaches 25,530 MiB. Attention begins after a flush and uses 12,434 MiB. The MLP holds eight matrices and reaches the block peak of 48,610 MiB. The largest live stage, rather than every encoded weight in the block, determines peak host memory.

6.4 Placement and parallelism

Plaintext preparation also depends on where host work runs. The retained implementation binds threads to physical cores and uses NUMA-local allocation so encoding threads stay close to the memory they allocate and to buffers staged for the GPU; decryption, exact nonlinear evaluation, and re-encryption use the same host resources. Template construction is then batched across the available 32 cores with dynamic scheduling, since each diagonal can be encoded independently without changing the order or result of encrypted multiplications; parallel and serial construction were checked for equal outputs before the parallel path was retained. Affinity controls placement and batching exposes independent diagonals to the cores, so together they make plaintext preparation a core-local parallel batch under the same cryptographic parameters. Their elapsed-time effect awaits a sourced paired measurement.

7 Results

The client-assisted circuit is evaluated along four axes: numerical agreement, executed work, memory, and complete-inference cost.

7.1 Evaluation setup and endpoints

We evaluate complete encrypted inference at the 103-token genomic-signal benchmark, the longest prompt among the encrypted-scope classification tasks. The complete run uses the first verified positive genomic-signal example, selected deterministically instead of by its prediction margin. Local tokenization and embedding occur before encryption; the encrypted endpoint contains all 12 transformer blocks and the task head.

The complete model is executed once on a whole-node allocation with one 40 GB A100 GPU and 32 CPU threads; the recorded telemetry shows no evidence of resource contention, although scheduler exclusivity was not requested. Timing starts before weight and reference loading and ends after final decryption. Server and client roles share one process and node, so boundary calls include cryptographic work but no serialization or network transport. That single execution supplies every complete-model timing below.

7.2 Correctness

Before encrypted evaluation, we fixed the relative-error acceptance threshold at 4×10^{-2} . As Section 4 describes, the NumPy float64 reference first agrees with an independent PyTorch float64 execution after every transformer block and the task head. The encrypted output is then compared with that checked NumPy reference; the original float32 implementation is recorded separately to distinguish dtype drift from CKKS error.

Table 4: Host-side optimizations that preserve the encrypted circuit. Effects report verified changes in executed work or memory; no controlled latency comparison is available.

Change	Verified effect	Status
Re-encode every plaintext per multiplication	159,744 encodings	baseline
Thread affinity, NUMA-local allocation	host placement changed	retained
Encode once, clone per use	removes 90% of encodings	retained
Parallel batched encoding	equal to serial encoding	retained
Encode at use level, flush between stages	peak 48,610 MiB RSS	enabling
Net	checked operation schedule unchanged	

Table 5: Checked operation schedule for the evaluated transformer block.

Operation	Count	Executor
Dense products	156	compute provider
Ciphertext–plaintext multiplications	177,734	compute provider
Ciphertext–ciphertext multiplications	1,506	compute provider
Rotations	8,173	compute provider
Slot reductions	8,776	compute provider
Physical client boundary crossings	857	data owner
Attention-related client calls	818 (95.4%)	data owner

For one block at the 103-token benchmark, the decrypted output has relative infinity-norm error 4.64×10^{-9} , leaving seven orders of magnitude of headroom. The worst token-relative error is 4.7×10^{-9} , and the largest absolute deviation is 7.44×10^{-8} . These values establish numerical agreement for the encrypted block with the DNAGPT weights and exact client nonlinearities.

The complete model is held to the same unchanged tolerance. Evaluating all 12 transformer blocks and the task head at the 103-token benchmark produces the same classification label as the plaintext reference. The relative classification-margin error is 8.56×10^{-9} , or 6.67 orders of magnitude below the predeclared threshold, and the absolute margin error is 1.05×10^{-7} . The encrypted, NumPy float64, and original float32 paths all produce label N; relative to the original float32 margin, the encrypted margin differs by 1.61×10^{-6} .

CKKS numerical error increases across blocks. The hidden-state relative infinity norm is 4.09×10^{-9} after the first block and 2.07×10^{-8} after the twelfth, while the largest absolute deviation grows from 6.56×10^{-8} to 7.97×10^{-6} , roughly two orders of magnitude across the model. The twelfth-block activation remains six orders of magnitude inside the acceptance tolerance. The nonlinear functions are evaluated exactly at client boundaries and therefore contribute no polynomial-approximation error.

7.3 Executed work

The counts locate the boundary cost in attention, as Section 5 sets out, and the benefit of token packing in the dense schedule: eight-token packing reduces the serial-equivalent dense-product count from 1,236 to 156, a $7.92\times$ reduction in operations. Rotations, masking, client work, and host preparation remain in the packed circuit, so that factor describes operation count rather than latency.

One block presents 130 ciphertext objects to the client and receives 766 in return. Across the complete model, ideal batching of independent calls within a protocol stage yields a lower bound of 65 sequential transport phases including input and output. The present implementation instead performs 10,299 in-process boundary calls. Ciphertext levels, serialization representation, compression, and transport overhead were not recorded, so a byte count cannot be derived honestly from the object count.

7.4 Memory

Device-wide GPU memory used during the complete run reaches 9839 MiB, within the 40 GB device. The GPU was empty at allocation, making attribution to the run reasonable, but the sampler does not expose a per-process memory metric. Host memory follows the stage lifetime of encoded plaintext templates measured in Section 6, peaking in the MLP stage of each block rather than accumulating across the block.

GPU memory reaches 9839 MiB during initial context and key loading and remains flat across the twelve transformer blocks. Host resident memory peaks at 50,930 MiB and repeats the same stage-level pattern in each block instead of accumulating. This behavior is consistent with the bounded multiplicative depth and stage-scoped plaintext cache used by the client-assisted protocol.

7.5 Composition and evaluation boundary

Two consecutive transformer blocks compose at reduced sequence length through a full-hidden-state client refresh, reaching 8.48×10^{-11} relative error after the second block, with no growth in device-wide GPU memory used. The complete model uses that mechanism throughout: all 12

Table 6: Derived ciphertext-object accounting for a complete inference. Counts follow from the checked protocol schedule; they are not observed network messages.

Direction or quantity	Count	Scope
Server to client	1,719	nonlinear and refresh boundaries
Client to server	9,339	nonlinear and refresh boundaries
Both directions	11,058	boundary ciphertext objects
Initial encrypted inputs	13	client to server
Final encrypted output	1	server to client
Total including endpoints	11,072	ciphertext objects
Minimum sequential phases	63	boundaries, with ideal same-stage batching
Minimum phases including endpoints	65	algorithmic lower bound
Serialized byte volume	unknown	no wire format was measured

Table 7: Encrypted evaluation completed at each reported scope. The 103-token benchmark is the longest prompt among the encrypted-scope classification tasks. Hidden-state error is a relative infinity norm over the activation; margin error is the scalar the classifier acts on. The two are different quantities and are kept in separate columns.

Scope	Status	Hidden-state error	Margin error
One block, 2 tokens	measured	—	—
Two consecutive blocks, 2 tokens, client refresh	measured	8.48×10^{-11}	—
One block, 103-token benchmark	measured	4.64×10^{-9}	—
Twelve blocks + task head, 103-token benchmark	measured	2.07×10^{-8}	8.56×10^{-9}
Encrypted token-index lookup	out of scope	—	—

Table 8: One complete encrypted inference: twelve transformer blocks and the task head at the 103-token benchmark, on one A100 GPU node using 32 CPU threads. Single execution on a whole-node allocation with no evidence of resource contention in the recorded telemetry; these are one sample’s values, not averages over repetitions. Shares are rounded independently and need not sum to 100%.

Phase	Seconds	Share of wall
Load DNAGPT weights and plaintext reference	23.494	0.4%
Crypto context, key generation, GPU key upload	3.945	0.1%
Encrypt the embedded input	1.265	0.0%
Encrypted evaluation	6594.748	98.7%
provider-side encrypted evaluation	5638.561	84.4%
declared client boundaries and refreshes (CPU)	956.187	14.3%
Final decrypt	0.044	0.0%
Uninstrumented remainder	59.504	0.9%
Wall clock	6683	

blocks and the task head run in one key lineage, separated by 11 full-hidden-state refreshes between consecutive blocks and one further refresh of the final token into the head. Fresh client encryption prevents multiplicative depth from accumulating across transformer blocks: the maximum required depth remains 13 throughout the twelve-block execution, and no homomorphic bootstrapping is used.

7.6 Cost of a complete encrypted inference

A complete 103-token benchmark takes 6683 s, or 1.86 hours. Encrypted evaluation accounts for 6594.748 s (98.7%) of wall time; setup and final decryption are negligible at this scale.

Within encrypted evaluation, the compute provider accounts for 85.5% and the data owner for 14.5% (Figure 4(c)); the data-owner share runs on CPU alone. The provider bucket includes the GPU backend and its host-side preparation, synchronization, and evaluator overhead.

The 1,755-token abundance-regression prompt is outside the encrypted scope and would induce a substantially larger causal-attention circuit under the current packing.

7.7 Feasibility

Under the stated client-assisted protocol, all 12 transformer blocks and the task head execute at the 103-token benchmark under one key lineage, with a maximum multiplicative depth of 13, bounded accelerator memory, and no homomorphic bootstrapping. The resulting prediction satisfies the predeclared numerical criterion. This establishes complete-model graph feasibility for the evaluated benchmark; encrypted task accuracy across additional examples remains outside the present evidence.

7.8 Practicality, judged separately

The complete 103-token benchmark takes 1.86 hours on one A100 and is therefore unsuitable for conventional interactive querying in its current form. The intended

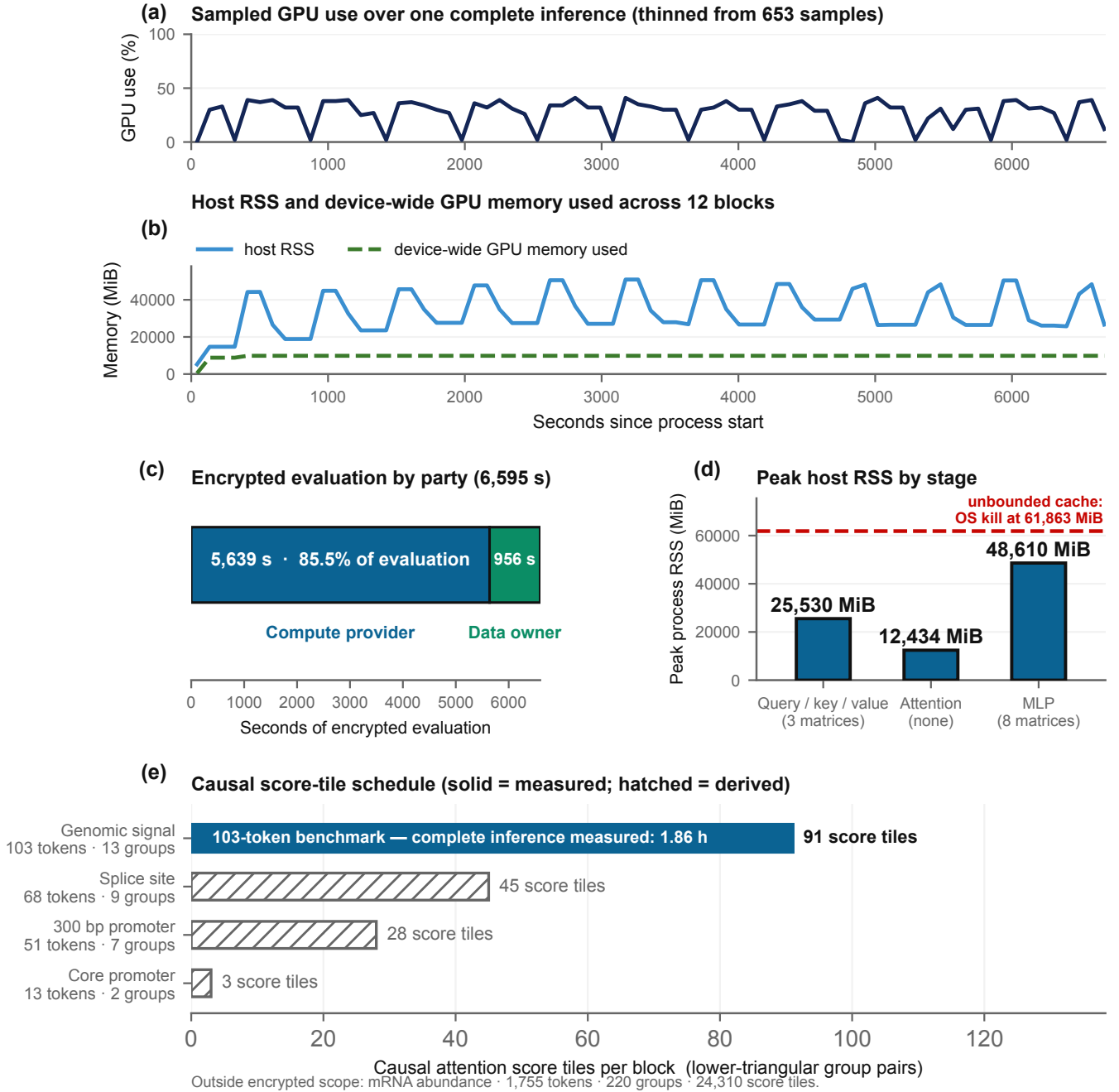


Figure 4: Systems behavior of the complete 103-token benchmark. (a) Sampled GPU utilization. (b) Host and device memory across all twelve transformer blocks; GPU memory is device-wide, not per-process. (c) The encrypted-evaluation interval divided between provider and data owner. Provider time includes GPU-backend work and its host support; data-owner time contains the CPU boundaries and refreshes. (d) Stage host memory for one block under the bounded plaintext cache, with the separate unbounded-cache attempt. (e) Derived causal score-tile schedules across the encrypted-scope tasks; only genomic-signal recognition has a measured complete inference.

deployment presents a different trade-off: a provider retains the model server-side while a data owner accepts additional latency to prevent a semi-honest provider from observing the genomic input or intermediate activations in plaintext. For low-volume, privacy-sensitive inference, such a trade-off may still be useful; this study does not establish production throughput or service-level practicality.

8 Related work

8.1 Private transformer inference

Private neural-network inference has long combined cryptographic tools according to the operation being evaluated. Gazelle established an influential hybrid design in which homomorphic encryption handles linear layers and garbled circuits handle nonlinearities [Juvekar et al. \[2018\]](#). Transformer systems inherited this decomposi-

tion but had to contend with attention, normalization, and activation functions at substantially greater scale. BOLT, Iron, Nimbus, and BumbleBee combine homomorphic encryption with secret sharing, oblivious transfer, and specialized two-party protocols for transformer inference Pang et al. [2024], Hao et al. [2022], Li et al. [2024], Lu et al. [2025]. Their protocols optimize a joint secure computation between parties; the present work instead keeps the server’s linear path in CKKS and assigns exact nonlinear evaluation to the data owner at explicit boundaries. This choice changes both the security contract and the allocation of work between the parties.

The client-aided division of labor predates transformer work. nGraph-HE2 evaluates standard pre-trained networks under CKKS while returning to the client to compute their native activation functions, precisely so that those activations need not be replaced by polynomials Boemer et al. [2019], and THE-X carries the pattern to transformers: the server evaluates HE-compatible operations and the client decrypts and re-encrypts around nonlinear functions Chen et al. [2022]. Safhire is the closest architectural antecedent Biswas et al. [2025]: it likewise places homomorphically evaluated linear layers on the server and has the client decrypt an intermediate, apply the nonlinearity in plaintext, and return a fresh encryption, though it studies convolutional networks and adds randomized output permutation to make reconstruction from released intermediates harder. The client-assisted pattern is thus established prior art, and we adopt rather than propose it. What this study adds is its application to a complete forward pass of a pretrained genomic transformer at the model’s task-defined prompt, preserving the original weights and exact nonlinearities, together with an independently checked plaintext reference that makes the resulting arithmetic, boundary schedule, and memory behavior reproducible.

The non-interactive line of work supplies useful cost context. THOR evaluates a 12-layer, 768-wide encoder at 128 tokens on a single A100, at ring degree 2^{16} , depth 13, and 128-bit security, in 602 s end to end, with no client interaction at any point Moon et al. [2025]. NEXUS reaches non-interactivity over RNS-CKKS with SIMD ciphertext compression and a secure argmax Zhang et al. [2025]; its headline per-inference figure is amortized over a batch and its single-prediction latency is substantially higher, so the two readings should not be conflated. MOAI reports a further 52.8% reduction in evaluation time against THOR by holding one packing convention across all layers and removing rotations from softmax and normalization Zhang et al. [2026]. Powerformer instead modifies the model, training HE-suitable polynomial components rather than preserving a pretrained transformer unchanged Park et al. [2025], and ELLMo treats packing and depth as coupled design choices Guzelhan et al. [2026].

Direct latency comparisons with these systems are not controlled because the evaluated models and cryptographic circuits differ. Our protocol preserves DNAGPT’s original nonlinear functions through client

evaluation and requires no homomorphic bootstrapping.

8.2 Homomorphic encryption for genomics

Homomorphic encryption already has a substantial genomic-computing lineage. The iDASH secure genome analysis competition evaluated encrypted association studies, sequence search, and related privacy-preserving workflows Kuo et al. [2020]. Private genomic-query systems combine homomorphic encryption with hashing and set-intersection techniques to test for variants without disclosing either the stored genome or the query Çetin et al. [2017]. Those workloads center on searches or prescribed statistics over genomic records. DNAGPT inference instead composes dense projections, attention, normalization, and a feed-forward network while preserving the numerical behavior of a pretrained foundation model. This study extends the genomic HE lineage to the feasibility boundary of that transformer forward pass.

The iDASH series has since moved from encrypted statistics toward encrypted neural evaluation, and supplies the closest prior art to this study. Its 2024 homomorphic-encryption track distributed DASHformer, a protein-family classifier comprising a tokenizer, an encoder, a single transformer block with four-head self-attention, and a dense classification layer, totaling 138,905 parameters over 50-residue fragments Harmanci et al. [2024]. The winning solution evaluates it non-interactively under leveled CKKS, with no bootstrapping and no client participation at any point, classifying a batch of 163 encrypted sequences in about 165 s at 128-bit security Chen et al. [2024]. On interaction that result is strictly stronger than the design evaluated here, which returns to the key holder at every nonlinearity. It reaches that strength under different constraints: one transformer block against twelve, a purpose-trained competition classifier against a 0.1-billion-parameter backbone, and nonlinearities replaced by data-driven non-polynomial fits rather than preserved. To our knowledge, no publicly available pretrained genomic foundation model has previously been evaluated under homomorphic encryption for a complete forward pass.

8.3 GPU acceleration and homomorphic matrix primitives

GPU CKKS libraries move number-theoretic transforms, key switching, rotations, and ciphertext arithmetic onto accelerators; the evaluated backend supplies those primitives and interoperates with the host cryptographic runtime Agulló-Domingo et al. [2025], Al Badawi et al. [2022]. Its API also shaped the available systems designs: the absence of ciphertext serialization prevented process-sharded evaluation, leaving thread-level concurrency and in-process device placement as the implementable paths. Recent multi-GPU transformer work treats placement and communication-computation overlap as first-class scheduling problems Gong et al. [2026],

but the current implementation neither serializes ciphertexts between processes nor exercises a networked transport, so those paths must be evaluated directly before adopting a multi-device schedule.

Matrix layout determines which rotations, plaintext encodings, and reductions an encrypted linear layer requires. THOR’s diagonal-major representation and compact packing specialize that layout for transformer inference Moon et al. [2025]; HE-BLAS develops a broader library of homomorphic linear-algebra kernels and shape-aware algorithms Bae et al. [2026]. These works motivate future work on packed transforms and attention reductions, as does MOAI’s rotation-free formulation of softmax and normalization Zhang et al. [2026]. Because packing, key material, level consumption, and the operation schedule interact, each candidate primitive must be revalidated against the plaintext reference and checked schedule before its performance is compared.

9 Limitations

Complete-model timing and numerical agreement are based on a single execution of the 103-token benchmark. This supports the reported execution cost and correctness comparison but not run-to-run variance, encrypted task accuracy, or input-domain numerical generality. The input was selected deterministically as the first positive genomic-signal record, not randomly or by prediction margin.

The protocol requires the key holder to remain available at the predetermined nonlinear and refresh boundaries. Availability and throughput therefore differ from non-interactive HE systems.

Client and server are roles inside one program on one node. Boundary calls perform the cryptographic operations directly, without ciphertext serialization, network transport, or a key custody service. A physical client boundary crossing is therefore not a network message. A derived schedule accounts for ciphertext objects and minimum dependency phases, but serialized byte volume, bandwidth sensitivity, transport latency, and production key handling remain unknown.

Encryption begins at embedded numeric vectors. Tokenization, token-index lookup in the embedding table, and construction of the initial embedding occur before the encrypted boundary. The evaluated deployment assumes that the tokenizer and embedding table are distributed to the client. Private lookup is required if a provider will not make those components available.

Model confidentiality is not a cryptographic goal of the protocol. The data owner observes intermediate activations at the client boundaries, which may expose information useful for model extraction. We do not evaluate such attacks; server-side model placement is an operational deployment choice rather than model protection.

CKKS decryption is approximate. The client never returns a decoded value to the server, only a freshly en-

cryptured result, but the server selects the ciphertext supplied to each declared boundary. Security analyses of approximate homomorphic encryption show why adversarially selected decryption queries require care Li and Micciancio [2021]. The semi-honest threat model rules out deviations from the declared schedule, and the current re-encryption path applies no noise flooding. Numerical headroom alone does not establish a secure flooding parameter. Selecting and validating that budget remains future work.

10 Conclusion

We show that client-assisted CKKS can execute all twelve DNAGPT transformer blocks and the task head from encrypted embedded inputs at the 103-token benchmark, the longest prompt among the encrypted-scope classification tasks. The predeclared relative-error tolerance was 4×10^{-2} ; the observed classification-margin error was 8.56×10^{-9} , and the prediction matched the plaintext reference. Client refresh keeps the maximum multiplicative depth at 13 and avoids homomorphic bootstrapping, with GPU memory remaining bounded across blocks.

The principal limitation is runtime: the complete benchmark requires 1.86 hours on one A100. This is unsuitable for conventional interactive inference, but it may represent an acceptable trade-off in low-volume settings where a provider retains a model server-side and a data owner prioritizes genomic confidentiality against a semi-honest provider. Improving provider-side encrypted throughput and evaluating a real networked deployment are the main steps toward practical use.

Future work

Future work should repeat complete-model evaluation across a predeclared set of genomic inputs and obtain repeated latency measurements. The main systems target is provider-side encrypted throughput, including additional plaintext caching and improved packed linear transforms. A deployable implementation must add serialized network transport and private embedding lookup. Stronger deployment security also requires a validated re-encryption and noise-flooding strategy.

Ethics statement

This study involved no human participants, specimens, or private clinical records. It used public reference datasets with recorded provenance. The work makes no clinical-performance claim and does not present the prototype as a deployable privacy guarantee beyond its stated threat model.

Data availability statement

All evaluation datasets are public. Their original sources, versions, preprocessing, and any recovery route used are recorded in the repository's data-provenance document. Sequence data, model weights, and cryptographic keys are not redistributed.

Code availability statement

The evaluation harnesses, encrypted-evaluation implementation, environment definitions, and the evidence record for the paper are maintained at https://github.com/Georgakopoulos-Soares-lab/private_genomic_dnagpt, released under the MIT license. Upstream model code, released weights, and evaluation datasets retain the licenses of their original distributors and are not redistributed here.

References

- V. Agarwal and J. Shendure. Predicting mrna abundance directly from genomic sequence using deep convolutional neural networks. *Cell Reports*, 31(7):107663, 2020. doi: 10.1016/j.celrep.2020.107663.
- C. Agulló-Domingo, O. Vera-López, S. Guzelhan, L. Daksha, A. El Jerari, K. Shivdikar, R. Agrawal, D. Kaeli, A. Joshi, and J. L. Abellán. FIDESlib: A fully-fledged open-source fhe library for efficient ckks on gpus. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software*, pages 365–367, 2025. doi: 10.1109/ISPASS64960.2025.00045. URL <https://arxiv.org/abs/2507.04775>.
- A. Al Badawi, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, I. Quah, Y. Polyakov, R. V. Saraswathy, K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca. OpenFHE: Open-source fully homomorphic encryption library. In *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 53–63, 2022. doi: 10.1145/3560827.3563379.
- Y. Bae, J. H. Cheon, G. Hanrot, J. H. Park, and D. Stehlé. Fast homomorphic linear algebra with BLAS. *Journal of Cryptology*, 39:25, 2026. doi: 10.1007/s00145-026-09580-x. Preprint: arXiv:2503.16080.
- S. Biswas, P. Chartier, A. Dhasade, T. Jurien, D. Kerriou, A.-M. Kerrmarch, M. Lemou, F. Tranie, M. de Vos, and M. Vujanovic. Practical and private hybrid ml inference with fully homomorphic encryption. *arXiv preprint arXiv:2509.01253*, 2025. URL <https://arxiv.org/abs/2509.01253>.
- F. Boemer, A. Costache, R. Cammarota, and C. Wierzynski. nGraph-HE2: A high-throughput framework for neural network inference on encrypted data. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 45–56, 2019. doi: 10.1145/3338469.3358944. URL <https://eprint.iacr.org/2019/947>.
- G. S. Çetin, H. Chen, K. Laine, K. Lauter, P. Rindal, and Y. Xia. Private queries on encrypted genomic data. *BMC Medical Genomics*, 10(Suppl 2):45, 2017. doi: 10.1186/s12920-017-0276-z.
- J. Chen, L. Yang, C. Yang, S. Wang, R. Li, W. Miao, W. Wu, L. Yang, K. Wu, and L. Dai. Secure transformer-based neural network inference for protein sequence classification. *Cryptology ePrint Archive*, Paper 2024/1851, 2024. URL <https://eprint.iacr.org/2024/1851>. First-place solution, iDASH 2024 Track 1.
- T. Chen, H. Bao, S. Huang, L. Dong, B. Jiao, D. Jiang, H. Zhou, J. Li, and F. Wei. THE-X: Privacy-preserving transformer inference with homomorphic encryption. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 3510–3520, 2022. doi: 10.18653/v1/2022.findings-acl.277. URL <https://aclanthology.org/2022.findings-acl.277/>.
- J. H. Cheon, A. Kim, M. Kim, and Y. Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437, 2017. doi: 10.1007/978-3-319-70694-8_15.
- Y. Erlich, T. Shor, I. Pe’er, and S. Carmi. Identity inference of genomic data using long-range familial searches. *Science*, 362(6415):690–694, 2018. doi: 10.1126/science.aau4832.
- European Parliament and Council of the European Union. Regulation (EU) 2016/679 of the european parliament and of the council. Official Journal of the European Union, 2016. URL <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- Z. Gong, R. Ran, F. Yao, and W. Wen. AEGIS: Scaling long-sequence homomorphic encrypted transformer inference via hybrid parallelism on multi-GPU systems, 2026. URL <https://arxiv.org/abs/2604.03425>. Accepted at ICS 2026.
- S. N. Guzelhan, L. Daksha, C. Agulló Domingo, G. Jonatan, J. Kim, J. L. Abellán, D. Kaeli, and A. Joshi. ELLMo: Packing- and depth-aware encrypted transformer inference. *IACR Cryptology ePrint Archive*, Report 2026/198, 2026. URL <https://eprint.iacr.org/2026/198>.
- M. Gymrek, A. L. McGuire, D. Golan, E. Halperin, and Y. Erlich. Identifying personal genomes by surname inference. *Science*, 339(6117):321–324, 2013. doi: 10.1126/science.1229566.
- M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang. Iron: Private inference on transformers. In *Advances in Neural Information Processing Systems*, volume 35, pages 15718–15731, 2022. doi: 10.52202/068431-1143.
- A. Harmanci, L. Chen, M. Kim, and X. Jiang. Descriptor: Benchmarking secure neural network evaluation methods for protein sequence classification (iDASH24). *IEEE Data Descriptions*, 1: 109–112, 2024. doi: 10.1109/ieeedata.2024.3482283.
- C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan. GAZELLE: A low latency framework for secure neural network inference. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 1651–1669, 2018. URL <https://www.usenix.org/conference/usenixsecurity18/presentation/juvekar>.
- M. Kalkatawi, A. Magana-Mora, B. Jankovic, and V. B. Bajic. Deepgsr: an optimized deep-learning structure for the recognition of genomic signals and regions. *Bioinformatics*, 35(7):1125–1132, 2019. doi: 10.1093/bioinformatics/bty752.
- T.-T. Kuo, X. Jiang, H. Tang, X. Wang, T. Bath, D. Bu, L. Wang, A. Harmanci, S. Zhang, D. Zhi, H. J. Sofia, and L. Ohno-Machado. iDASH secure genome analysis competition 2018: blockchain genomic data access logging, homomorphic encryption on GWAS, and dna segment searching. *BMC Medical Genomics*, 13(Suppl 7):98, 2020. doi: 10.1186/s12920-020-0715-0.
- B. Li and D. Micciancio. On the security of homomorphic encryption on approximate numbers. In *Advances in Cryptology – EUROCRYPT 2021*, pages 648–677, 2021. doi: 10.1007/978-3-030-77870-5_23. URL <https://eprint.iacr.org/2020/1533>.
- Z. Li, K. Yang, J. Tan, W.-j. Lu, H. Wu, X. Wang, Y. Yu, D. Zhao, Y. Zheng, M. Guo, and J. Leng. Nimbus: Secure and efficient two-party inference for transformers. In *Advances in Neural Information Processing Systems*, volume 37, pages 21572–21600, 2024. doi: 10.52202/079017-0680.
- W.-j. Lu, Z. Huang, Z. Gu, J. Li, J. Liu, C. Hong, K. Ren, T. Wei, and W. Chen. Bumblebee: Secure two-party inference framework for large transformers. In *Network and Distributed System Security Symposium*, 2025. doi: 10.14722/ndss.2025.230057.
- J. Moon, D. Yoo, X. Jiang, and M. Kim. THOR: Secure transformer inference with homomorphic encryption. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 3765–3779, 2025. doi: 10.1145/3719027.3765150. URL <https://eprint.iacr.org/2024/1881>.
- S. Ouaari, J. Kreuer, and N. Pfeifer. How private are DNA embeddings? inverting foundation model representations of genomic sequences, 2026. URL <https://arxiv.org/abs/2603.06950>.

- Q. Pang, J. Zhu, H. Möllering, W. Zheng, and T. Schneider. BOLT: Privacy-preserving, accurate and efficient inference for transformers. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4753–4771, 2024. doi: 10.1109/SP54263.2024.00130.
- D. Park, E. Lee, and J.-W. Lee. Powerformer: Efficient and high-accuracy privacy-preserving language model with homomorphic encryption. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11090–11111, 2025. doi: 10.18653/v1/2025.acl-long.543. URL <https://aclanthology.org/2025.acl-long.543/>.
- D. Zhang, W. Zhang, Y. Zhao, J. Zhang, B. He, C. Qin, and J. Yao. Dnagpt: A generalized pre-trained tool for versatile dna sequence analysis tasks. *arXiv preprint arXiv:2307.05628*, 2023. URL <https://arxiv.org/abs/2307.05628>.
- J. Zhang, X. Yang, L. He, K. Chen, W.-j. Lu, Y. Wang, X. Hou, J. Liu, K. Ren, and X. Yang. Secure transformer inference made non-interactive. In *Network and Distributed System Security Symposium (NDSS)*, 2025. doi: 10.14722/ndss.2025.230868. Also IACR ePrint 2024/136.
- L. Zhang, X. Wang, J. J. Sim, Z. Huang, J. Zhong, H. Wang, P. Duan, and K.-Y. Lam. MOAI: Module-optimizing architecture for non-interactive secure transformer inference. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=qJn4HtTzhH>.
- Z. Zhou, Y. Ji, W. Li, P. Dutta, R. V. Davuluri, and H. Liu. Dnabert-2: Efficient foundation model and benchmark for multi-species genomes. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=oMLQB4EZE1>.